Institute of Business Administration Karachi
Leadership and Ideas for Tomorrow

Institute of Business Administration, Karachi

Department of Computer Science

Chess Game using C++ and SFML

Course:	Object Oriented Programming
Semester:	Spring 2026
Language:	C++17
Graphics Library:	SFML 3.0
Instructor:	Sir Behraj Khan

Submitted By

Jiya Turshani
Laksh Kumar
Deepak Raj

Department of Computer Science
Institute of Business Administration (IBA), Karachi

GitHub Repository:
`https://github.com/deepakvalmani/Chess-main.git`

May 10, 2026

Contents

1 Abstract

This project presents a complete Chess Game developed using modern C++ and the SFML graphics library. The project was developed as part of the Object Oriented Programming course at the Institute of Business Administration (IBA), Karachi.

The application implements major chess mechanics including move validation, legal movement generation, check detection, checkmate detection, stalemate detection, pawn promotion, and an interactive graphical user interface.

The project demonstrates practical implementation of Object Oriented Programming concepts including:

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction
- Dynamic Binding
- Composition
- Smart Pointers

The game supports both Human vs Human and Human vs AI gameplay modes.

2 Introduction

Chess is one of the most strategic and widely played board games in the world. Developing a chess game provides an excellent opportunity to apply advanced programming concepts and software architecture principles.

The primary goal of this project was to create a fully interactive graphical chess game using modern C++ and SFML while implementing Object Oriented Programming concepts learned during the course.

The complete system was designed from scratch including:

- Chess board implementation
- Piece movement logic
- Move validation
- Check and checkmate detection
- AI opponent
- Graphical User Interface

3 Objectives

The main objectives of this project are:

1. Implement a complete chess game.
2. Apply Object Oriented Programming concepts.

3. Design modular and reusable software architecture.
4. Implement legal chess mechanics.
5. Develop an AI opponent.
6. Create a graphical interface using SFML.

4 Project Features

4.1 Game Modes

4.1.1 Play vs AI

- Human player plays as White.
- AI plays as Black.
- AI generates random legal moves.

4.1.2 Play with Friend

- Local multiplayer support.
- Alternating turns between White and Black.

5 Implemented Chess Mechanics

Feature	Status
Pawn Movement	Implemented
Rook Movement	Implemented
Knight Movement	Implemented
Bishop Movement	Implemented
Queen Movement	Implemented
King Movement	Implemented
Capturing	Implemented
Move Validation	Implemented
Check Detection	Implemented
Checkmate Detection	Implemented
Stalemate Detection	Implemented
Pawn Promotion	Implemented
AI Opponent	Implemented

Table 1: Implemented Chess Mechanics

6 Object Oriented Programming Concepts

6.1 Inheritance

All chess pieces inherit from a common base class called `Piece`.

```
class Piece
{
protected:
    bool isWhite;
```

```
};
```

Derived classes include:

- Pawn
- Rook
- Knight
- Bishop
- Queen
- King

6.2 Polymorphism

Each chess piece overrides the movement function:

```
virtual std::vector<sf::Vector2i>
getValidMoves(...) const;
```

This allows dynamic move generation depending on piece type.

6.3 Encapsulation

Each class manages its own:

- Data
- Movement logic
- Rendering logic
- Internal state

6.4 Composition

The `Board` class contains all chess pieces using:

```
std::unique_ptr<Piece> grid[8][8];
```

6.5 Smart Pointers

The project uses:

```
std::unique_ptr
```

Benefits include:

- Automatic memory management
- Prevents memory leaks
- Safer ownership handling

7 Technologies Used

Technology	Purpose
C++17	Core Programming Language
SFML 3.0	Graphics and Rendering
CMake	Build System
Visual Studio	Development IDE
MSYS2	Compiler Environment

Table 2: Technologies Used

8 Project Structure

```
Chess/

 assets/
 include/
 src/
 CMakeLists.txt
 CMakeSettings.json
 README.md
```

9 Setup Instructions

9.1 Requirements

- CMake 3.14+
- MSYS2
- SFML 3.0
- Visual Studio 2022

9.2 Installing MSYS2

1. Download MSYS2 from:

   `https://www.msys2.org/`

2. Install using the UCRT64 environment.
3. Run:

```
pacman -S mingw-w64-ucrt-x86_64-sfml
```

9.3 Build Steps

9.3.1 Step 1: Create Build Folder

```
mkdir build
cd build
```

9.3.2 Step 2: Configure

```
cmake ..
```

9.3.3 Step 3: Build

```
cmake --build .
```

9.3.4 Step 4: Run

```
./Chess.exe
```

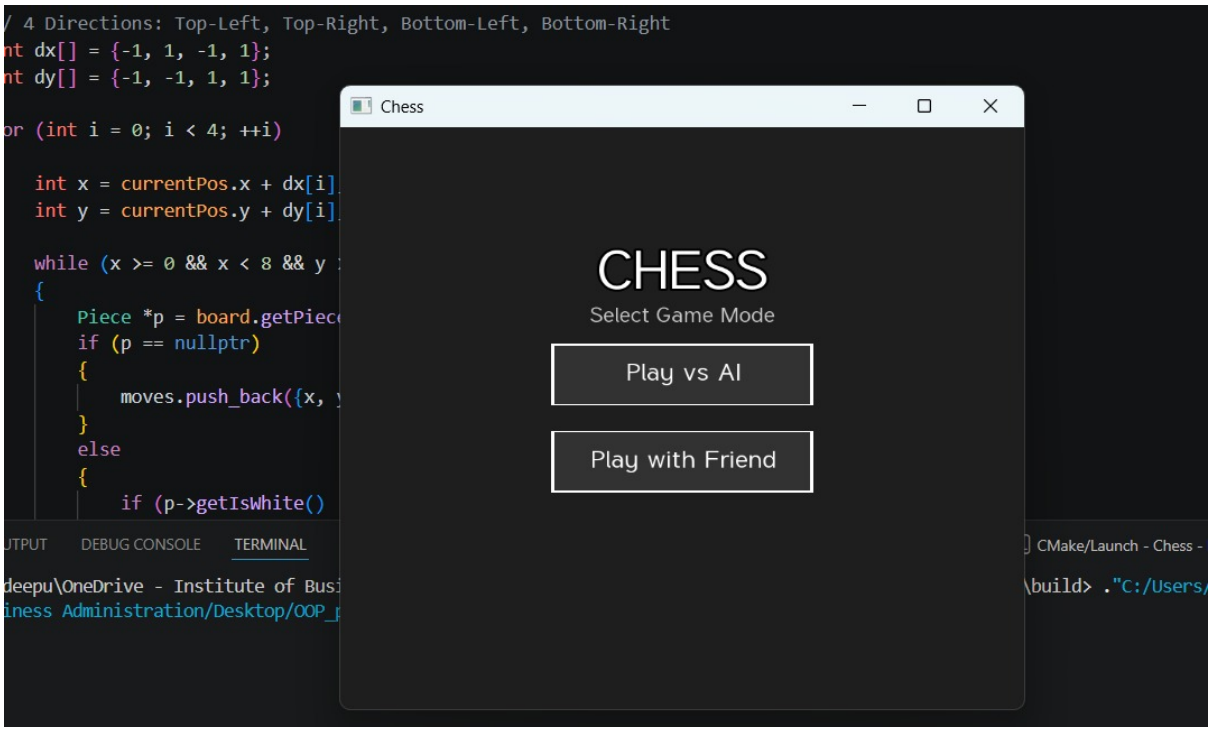

Figure 1: Initial screen

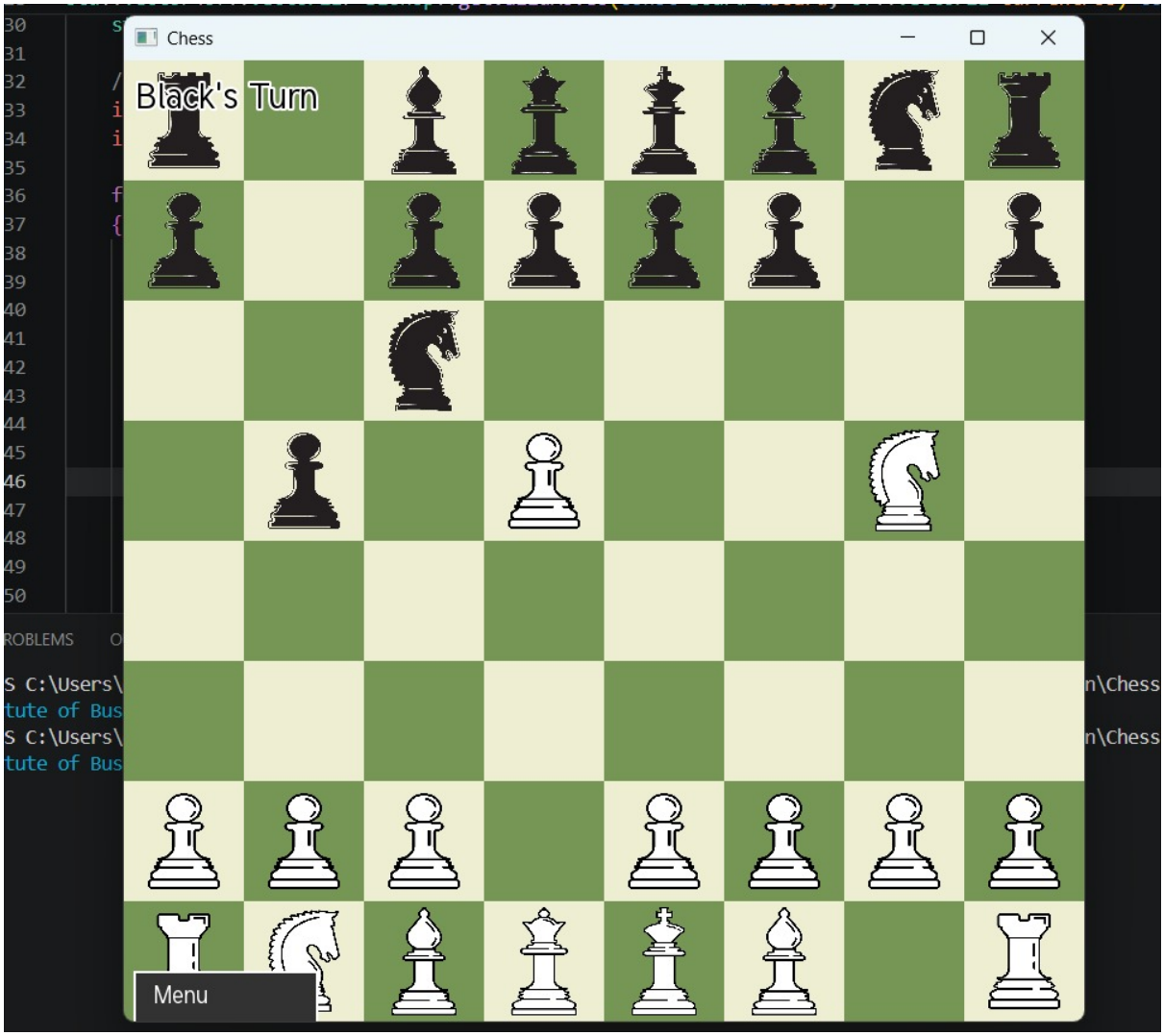

Figure 2: Game Screen

10 Game Controls

Action	Mouse Input
Select Piece	Left Click
Move Piece	Right Click
Return to Menu	Click Menu Button

Table 3: Game Controls

11 Game Logic

11.1 Piece Selection

The user selects a piece using left-click.

The game:

1. Detects the selected piece

2. Generates possible moves
3. Filters illegal moves
4. Highlights valid moves

11.2 Move Validation

Each piece first generates pseudo-legal moves.

The board then verifies:

- Move is inside the board
- Destination is valid
- Move does not expose the king to check

11.3 Check Detection

The system:

1. Finds the king position
2. Generates enemy attack moves
3. Checks whether the king is attacked

11.4 Checkmate Detection

A player is in checkmate when:

- The king is in check
- No legal moves exist

11.5 AI System

The AI:

1. Collects all legal moves
2. Stores them in a vector
3. Randomly selects one move
4. Executes the move

Future improvements may include:

- Minimax Algorithm
- Alpha-Beta Pruning
- Evaluation Functions

12 GUI Features

The graphical interface includes:

- Interactive chess board
- Piece textures

- Highlighted valid moves
- Status messages
- Turn indicators
- Menu system

13 Challenges Faced

During the development of this chess game, several technical and design challenges were encountered. These challenges helped in improving problem-solving skills and understanding of software architecture and object-oriented programming concepts.

13.1 Implementing Chess Movement Rules

One of the major challenges was implementing accurate movement logic for each chess piece. Every piece in chess follows different movement rules:

- Pawns move forward but capture diagonally.
- Knights move in an "L" shape and can jump over pieces.
- Bishops move diagonally across unlimited squares.
- Rooks move horizontally and vertically.
- Queens combine rook and bishop movement.
- Kings move only one square in any direction.

Designing separate movement algorithms for each piece while maintaining clean object-oriented design required careful planning and testing.

13.2 Check and Checkmate Detection

Implementing check detection was one of the most difficult parts of the project. The system needed to determine whether the king was under attack after every move.

To solve this problem:

- The board scans all enemy pieces.
- Each enemy piece generates its possible attack moves.
- The program checks whether any attack square matches the king's position.

Checkmate detection was even more complex because the game had to verify whether the player still had at least one legal move available to escape the check.

13.3 Move Validation

Another difficult task was preventing illegal moves. A move might appear mathematically correct but could expose the player's king to danger.

To handle this issue:

- The move is temporarily simulated in memory.
- The board checks whether the king becomes exposed.

- If the king is still safe, the move is allowed.
- Otherwise, the move is rejected.

This required careful manipulation of piece positions using smart pointers.

13.4 Managing Dynamic Memory

Since chess pieces are created dynamically, memory management was an important challenge. Improper handling could cause memory leaks or crashes.

The project uses:

`std::unique_ptr`

to ensure automatic memory management and safe ownership transfer during captures and movement.

13.5 SFML Rendering and GUI Design

Creating the graphical interface using SFML also introduced challenges such as:

- Correctly positioning pieces on the board
- Scaling textures properly
- Handling mouse input events
- Highlighting valid moves
- Managing menus and game states

Special attention was required to synchronize the visual board with the internal game logic.

13.6 Debugging Complex Game Logic

Debugging chess logic was challenging because many features depend on one another. A small issue in movement logic could affect:

- Check detection
- Capturing
- AI behavior
- Checkmate validation

Extensive testing and debugging were required to ensure stable gameplay.

14 Future Improvements

Potential future features include:

- Castling
- En Passant
- Stronger AI
- Sound effects
- Online multiplayer

- Move history
- Undo system
- Saving/loading games

15 Known Limitations

- AI uses random move generation
- Castling not implemented
- En passant not implemented
- No networking support

16 Learning Outcomes

This project helped improve understanding of:

- Object Oriented Programming
- Game development architecture
- GUI programming with SFML
- Event-driven programming
- Memory management
- Chess engine logic

17 Conclusion

This project successfully demonstrates the implementation of a complete Chess Game using modern C++ and SFML.

The application effectively applies Object Oriented Programming concepts and demonstrates practical software engineering, GUI programming, and game logic implementation.

18 References

- SFML Documentation:

  `https://www.sfml-dev.org/documentation/`

- C++ Reference:

  `https://cplusplus.com/`

- Chess Rules:

  `https://www.chess.com/learn-how-to-play-chess`